BCS5031 / SITARE UNIVERSITY / 5TH SEMESTER

Creative Problem Solving

Modern Software Engineering: Architecture, Operations, and AI-Native Development

T0 · L0 · ENGINEERING MINDSET

What you've built. What actually runs software.



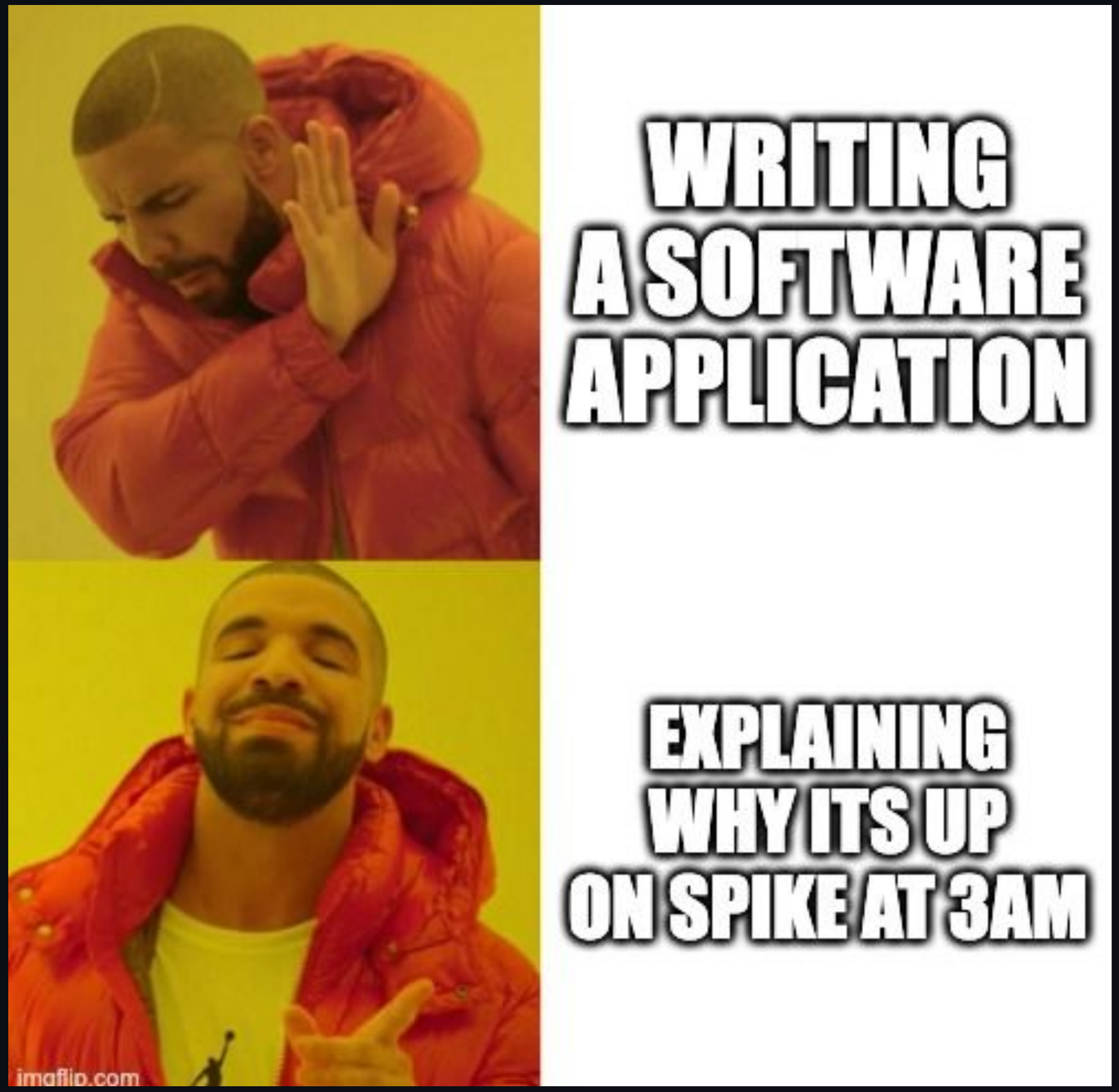
Your project.



What runs it in production.

Writing code is no longer the hard part.

Operating it always was.


WRITING A SOFTWARE APPLICATION
EXPLAINING WHY ITS UP ON SPIKE AT 3AM
imgflip.com

Same code. Two very different jobs.

Writing Code

- AI makes syntax generation trivial
- The blank page stopped being the obstacle

Operating Software

- Keeping unfamiliar systems alive under load
- Deploying without breaking production
- Managing databases without destroying user data

The Startup-Stage Decision

Four questions you'll answer with data, not fashion.

T0 · L0 / THE CORE DECISION

Every project starts with the same four questions.

01 What stack will you use?

02 How will you run it?

03 Where will you host it?

04 What will it cost?

Every answer is driven by **data and system constraints** — never by developer fashion.


USE WHAT THE CONSTRAINTS NEED
USE WHAT THE YOUTUBER USED
YOU
imgflip.com

What Is The Need For This Course?

Two outcomes. Both compound weekly, whether you notice or not.

T0 · L0 · ENGINEERING MINDSET

Outcome 1: Technical Resilience

- Premature distribution
- Unmonitored endpoints
- Hosting bills nobody modeled
- Databases touched with no rollback plan

Predictable mistakes. They kill early-stage products every time.

T0 · L0 · ENGINEERING MINDSET

Outcome 2: Employability From the First Fortnight

Fifth-semester, placement season. This course compresses the first two years of a junior engineer's practical experience into a single semester.

That gap is what separates a graduate from a hire.


ALL THE BEST
2 YRS OF ENGINEERING
IN ONE SEMESTER
imgflip.com
THUMBS-AMMO

How You're Graded

Consistency is mandatory. Cramming is not an option.

T0 · L0 / ASSESSMENT

Eighty percent of your grade is earned weekly.



80% — Continuous assessment
Milestones & quizzes, every week

20% — Capstone project
One project, at the end

No single exam saves the semester. No single exam sinks it, either.

The Course Map

T0
Professional Loop & Version Control

T1
AI-Native Workflow

T2
Testing & CI · Milestone 1

T3
Containers & DB Migrations

T4
Architecture & Stack Decision

T5
Continuous Delivery

T6
Performance & Scaling · Milestone 2

T7
Observability & Cloud Economics

T8
Security 101

T9
Agentic & LLM Security · Milestone 3

T10
Capstone Project

Where Are You Starting From?

https://tinyurl.com/3fs25fxj



T0

These weeks set the standard for everything after.

- Toolchain & the professional loop
- AI-native workflows
- Version control mechanics & continuous integration

WEEKS 4–6 / WHAT DOESN'T UNDO ITSELF

Code is reversible. Data often isn't.

- Containerization
- Database schema evolution
- Architecture decisions — monolith vs. microservices
- Safe continuous delivery

WEEKS 7–10 / UNDER PRESSURE

You'll defend this system live.

- Performance & scaling
- Observability with OpenTelemetry
- Classic security threats
- Defending AI agents against prompt injection

The capstone: **your system, your defense**, in front of the room.

Team Structure & the Rotating Lead

LEAD · 2-WK WINDOW

Merges PRs

Member

Member

Member

Only the nominated lead can merge pull requests during their designated window.

Ground Rules for AI

Part of the toolchain. Never a substitute for understanding.

T0 · L0 / ACADEMIC INTEGRITY

Where AI is allowed. Where it isn't.

Permitted

- All practical work and every milestone
- Using it well is taught and assessed

Not Permitted

- Quizzes
- Individual exercises
- The capstone defense

These three exist to measure **what can't be delegated**.

Who is accountable for this code?

You. Every time.

T0 · L0 / ACADEMIC INTEGRITY

Never paste this into a prompt.

Production credentials

Personal data

Treated as an academic integrity failure - not a mistake.

A credential exposed this way must be **rotated, and the incident discussed** - exactly as industry treats it.

T0 · L0 / ACADEMIC INTEGRITY

The cost of getting this wrong.

25%

of the affected grade — charged to the **whole team** when one member defaults

Termination

from the course, for policy violations — **without exception**

Flagging a mistake early is treated **far more leniently** than one discovered later.


production
secrets are now public
THIS IS FINE.
imgflip.com

Two ways to store this key.

```
# vulnerable
API_KEY = "sk-live-4f9a2b8e1c3d"

# safe
API_KEY = os.environ["STRIPE_API_KEY"]
```

> One of these should never exist in a repository you've pushed.

Two ways to store this key.

```
# vulnerable
API_KEY = "sk-live-4f9a2b8e1c3d"

# safe
API_KEY = os.environ["STRIPE_API_KEY"]
```

> Hardcoded into source means **hardcoded into git history** — forever.

Two ways to store this key.

```
# vulnerable
API_KEY = "sk-live-4f9a2b8e1c3d"

# safe
API_KEY = os.environ["STRIPE_API_KEY"]
```

> Read from the environment. **Never written to a file git can see.**

Your Local Environment

Four tools. Every machine agrees on all four.

T0 · L0 · ENGINEERING MINDSET

The Local Toolchain

Git

Docker Compose

Managed Postgres

Claude Code

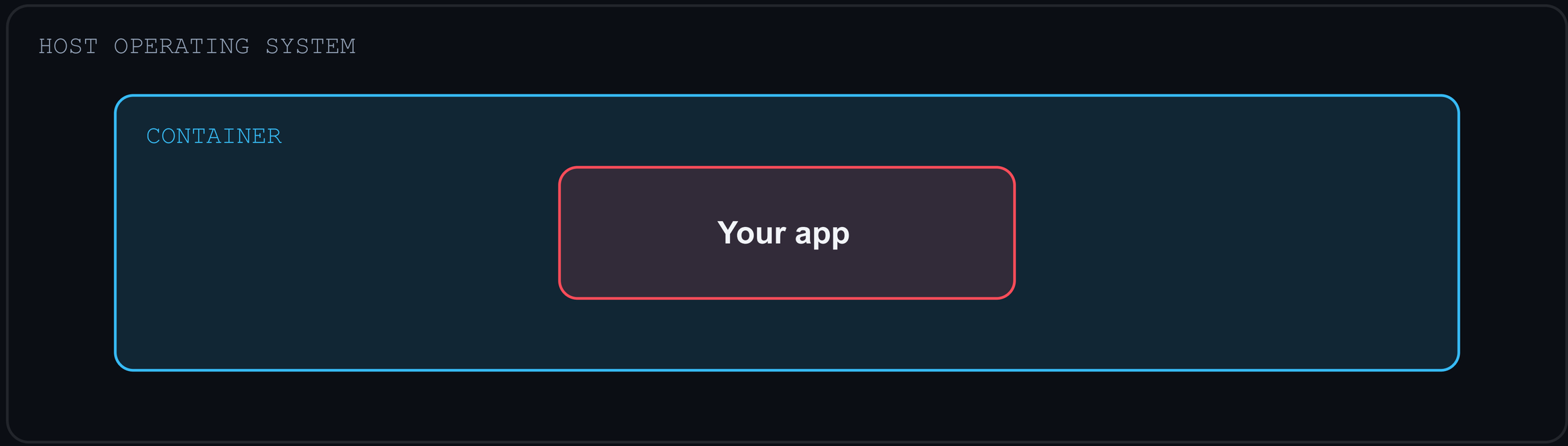
Same four tools, every team, all semester.

"It works on my machine."

Nobody Cares!

T0 · L0 / CONTAINERS

The container is the box, not the whole machine.



One command, everyone's machine agrees.

you — project — 80×24

```
$ docker compose up
Creating network "project_default"
Creating project_db_1 ... done
Creating project_app_1 ... done
db_1 | database system is ready to accept connections
app_1 | listening on 0.0.0.0:8000
✓ db healthy
✓ app healthy
```

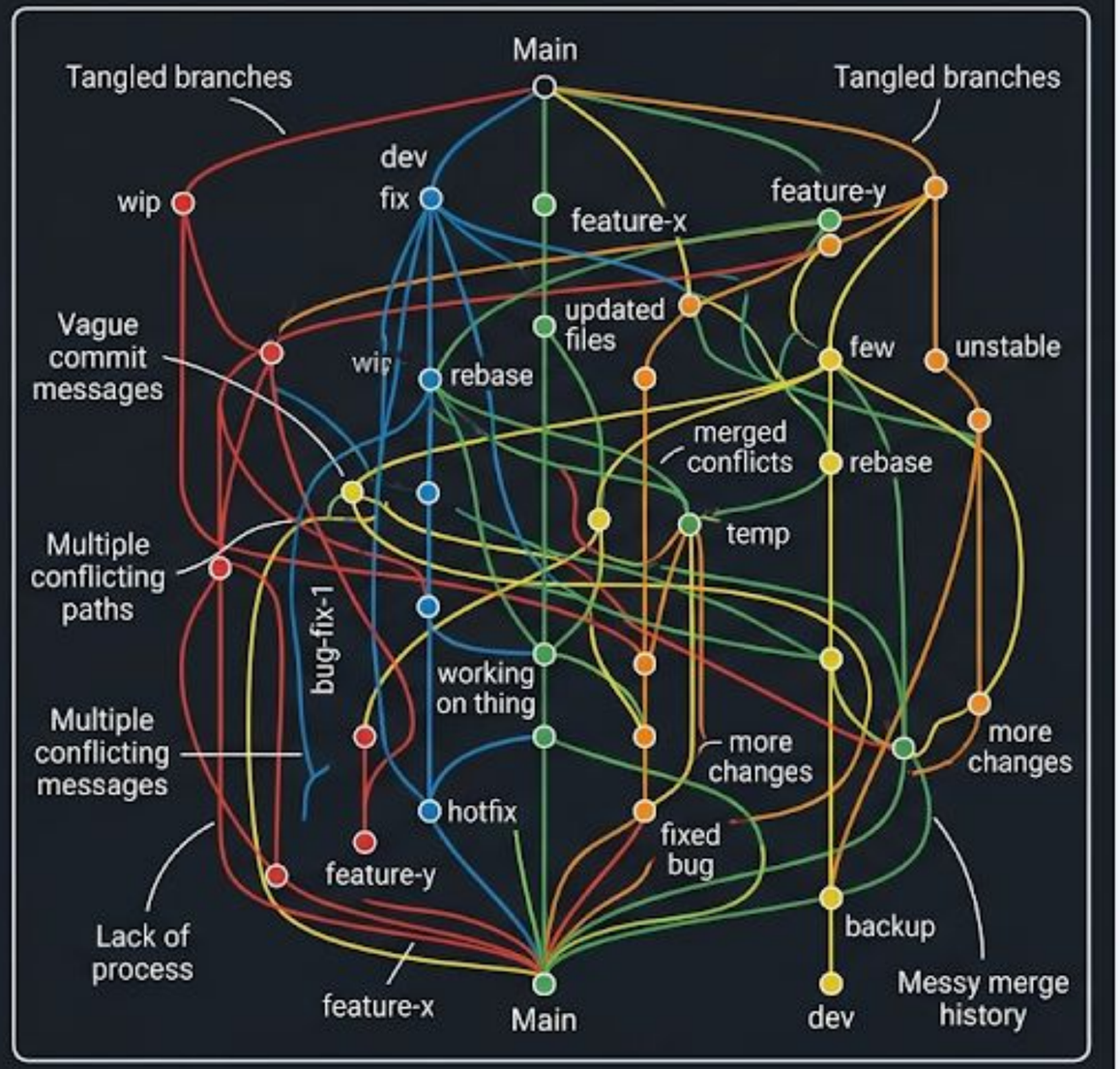
Version Control as Communication

Read by your teammates far more often than by you.

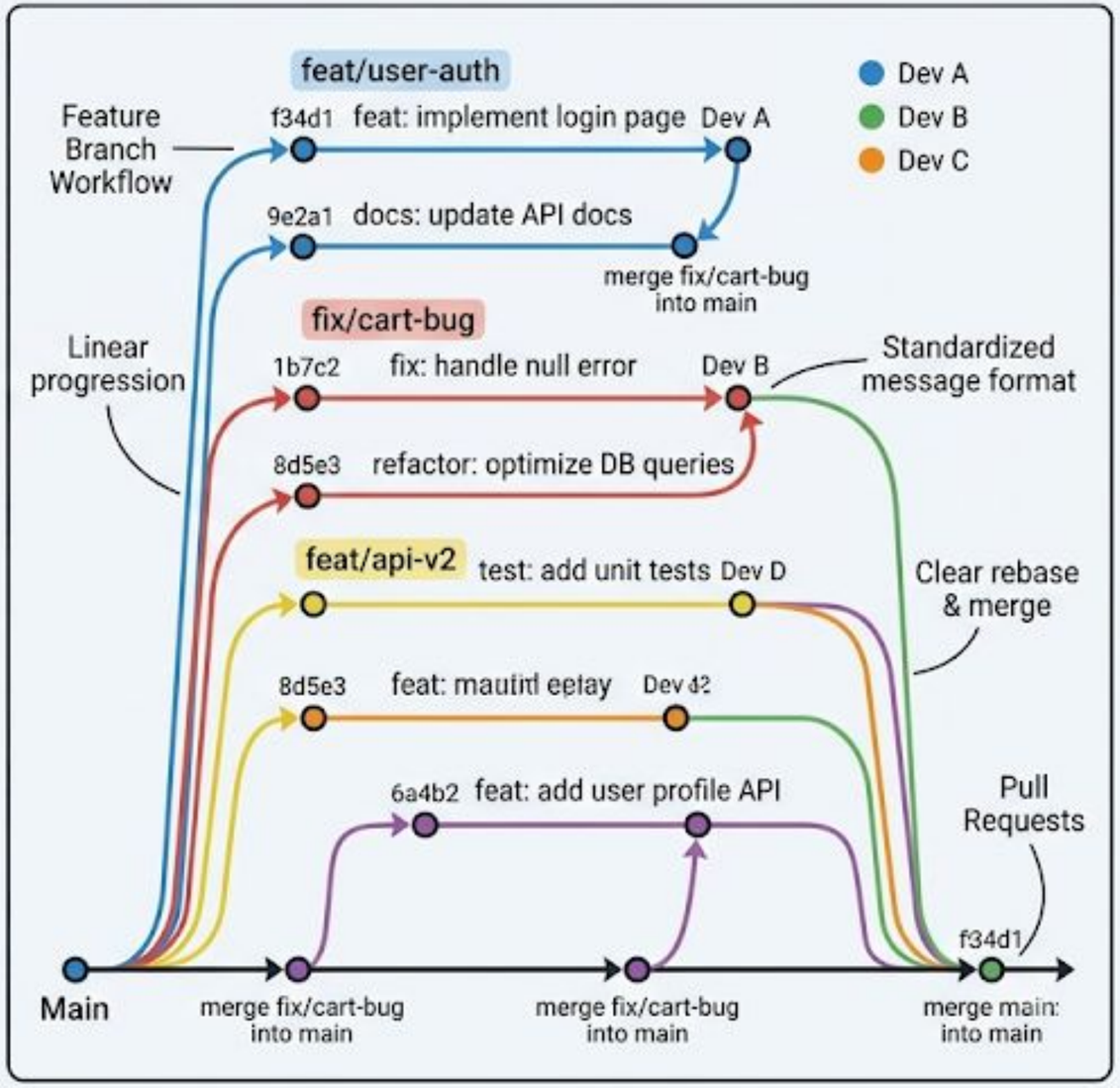
Is git a save button, or a conversation?

A conversation. Read by people.

CHAOTIC COMMIT HISTORY
(Unstructured Collaboration)



CLEAN COMMIT HISTORY
(Structured Collaboration & Conventional Commits)



A commit message has two jobs.

```
fix(auth): reject expired refresh tokens

Refresh tokens issued before the key rotation
were still being accepted after revocation.
Reject anything signed with the retired key
instead of trusting the expiry claim alone.
```

> Read it cold, top to bottom, before we focus on anything.

A commit message has two jobs.

```
fix(auth): reject expired refresh tokens

Refresh tokens issued before the key rotation
were still being accepted after revocation.
Reject anything signed with the retired key
instead of trusting the expiry claim alone.
```

> One imperative subject line: **"reject"**, not "rejects" or "rejected".

A commit message has two jobs.

```
fix(auth): reject expired refresh tokens

Refresh tokens issued before the key rotation
were still being accepted after revocation.
Reject anything signed with the retired key
instead of trusting the expiry claim alone.
```

> The body explains **why** — the diff already shows what.

T0 · L0 / COMMIT ANATOMY

One logical change. Every time.

```
fix(auth): reject expired refresh tokens

Refresh tokens issued before the key rotation
were still being accepted after revocation.
Reject anything signed with the retired key
instead of trusting the expiry claim alone.
```

> If the subject needs **"and"**, it's two commits.

The Pull Request Workflow

Every merge starts as a request for feedback.

T0 · L0 / PULL REQUESTS

Every merge starts as a request for feedback.

feat: rotate refresh tokens on key revocation — 2 approvals needed

3 files changed

auth/tokens.py	+28 −6
auth/tests/test_tokens.py	+14 −5
auth/middleware.py	+0 −0

R — Looks right — but check the grace-period edge case before you merge.

Approve · Request changes

Cross a bridge no one inspected?

Then don't merge code no one reviewed.

T0 · L0 / CODE REVIEW

Four things a reviewer is actually checking.

- **Correctness**
  Does it do what it claims to do
- **Blast radius**
  What breaks if this is wrong
- **Test coverage**
  Proof, not promises
- **Cognitive complexity**
  Can the next person actually follow this

DELIVERABLE

First Deliverable

01 Register your team

02 Open a pull request on the practice repository

03 Write a commit message that means something

04 Get it peer-reviewed, then merged

This is graded

Q/A

Questions

01 Isn't ops/DevOps a separate team's job in industry?

02 AI can already do deployment for me - why learn it by hand?

03 What if a teammate doesn't contribute?

04 What exactly counts as 'discussing approaches' vs. submitting another team's work?

05 We already have Docker/cloud experience from an internship - can we test out?

06 What actually happens if we accidentally commit a secret?

T0 · L0 / RECAP

What changed today.

- From "does it run" to "does it survive"
- From solo commits to reviewed history
- From guessing costs to modeling them
- From trusting AI's code to being accountable for it

Week one starts now.

Register your team. Open your first pull request. We'll see what breaks.